T13: TESTING AND DEBUGGING EVIDENCE

Weight: /4 — Requirement: at least three issues diagnosed with a suitable tool, corrected, and retested.

What a trace table is (in plain terms)

A trace table is just a chart where you run the code by hand, one line at a time, and write down what every variable equals at each step. It turns "I think this works" into "I checked it and here is exactly where it breaks." That written table IS your debugging evidence — it's the "suitable tool" the rubric is asking for.

Below are three real issues found in our own pseudocode (T9 and T10), each traced, fixed, and retraced to confirm the fix.

Issue 1 — Late submissions get no status colour

Where: updateAssignmentColors() — T10 pseudocode

Tool used: Manual trace table (desk check)

The function has three checks: not submitted + overdue → RED, submitted on time → GREEN, not submitted + not due yet → GREY. But it never checks "submitted LATE" (isSubmitted = true AND submitDate is AFTER dueDate). Tracing a late assignment through the function shows the bug:

Step	Condition checked	Result of check	statusColor after this line
1	isSubmitted==false AND dueDate<currentDate	FALSE (it WAS submitted)	unchanged
2	isSubmitted==true AND submitDate<=dueDate	FALSE (submitted AFTER dueDate)	unchanged
3	isSubmitted==false AND dueDate>=currentDate	FALSE (it WAS submitted)	unchanged
4	END IF reached — no branch matched	—	NEVER SET (bug)

Diagnosis: a late submission falls through all three IF/ELSE IF branches and statusColor is left blank. On the real app this would show as an assignment with no colour tag at all — a learner who handed work in late looks identical to one who hasn't touched it.

Fix: Add a fourth branch: ELSE IF assignment.isSubmitted == true AND assignment.submitDate > assignment.dueDate THEN assignment.statusColor = "ORANGE" // Late

Retest — same late assignment traced again after the fix:

Step	Condition checked	Result of check	statusColor after this line
1	isSubmitted==false AND dueDate<currentDate	FALSE	unchanged
2	isSubmitted==true AND submitDate<=dueDate	FALSE	unchanged
3	isSubmitted==false AND dueDate>=currentDate	FALSE	unchanged
4 (new)	isSubmitted==true AND submitDate>dueDate	TRUE	"ORANGE" ✔️ correct

Issue 2 — failedAttempts never resets between users

Where: login() — T9 pseudocode

Tool used: Manual trace table (desk check)

currentUser is declared as a global variable, but failedAttempts is used inside login() and is never declared or reset anywhere. Tracing two different people logging in one after another on the same session shows the problem:

Step	Who is logging in	Action	failedAttempts value
1	Learner A	wrong password	1
2	Learner A	wrong password	2
3	Learner A	wrong password → 3rd fail	3 → shows "Forgot password?"
4	Learner B (different person, correct password)	logs in successfully	still 3 (never reset)
5	Learner B	logs out, Learner C tries once, wrong password	4 (bug: counts from A, not C)

Diagnosis: failedAttempts is shared and never reset to 0, either when a login succeeds or when a new person starts typing. It keeps counting across completely different users, so the "3 wrong attempts" warning becomes meaningless — it can trigger after just one wrong attempt if someone else already failed twice.

Fix: Declare failedAttempts = 0 as a global next to currentUser, and add SET failedAttempts = 0 inside the IF branch where login succeeds.

Retest — same sequence traced again after the fix:

Step	Who is logging in	Action	failedAttempts value
1	Learner A	wrong password	1
2	Learner A	wrong password	2
3	Learner A	wrong password → 3rd fail	3 → shows "Forgot password?"
4	Learner B	logs in successfully	reset to 0 ✔️
5	Learner C	wrong password	1 ✔️ correct, starts fresh

Issue 3 — boundary values in generateProgressStatement

Where: generateProgressStatement() — T10 pseudocode

Tool used: Manual trace table (boundary testing)

The flag condition is: IF attendanceRate < 70 OR assignmentAverage < 50 THEN "NEEDS EXTRA HELP". Boundary values (exactly 70, exactly 50) are the classic place bugs hide, so we traced them on purpose:

Step	attendanceRate	assignmentAverage	attendanceRate<70?	assignmentAverage<50?	flag result
1	69	80	TRUE	FALSE	NEEDS EXTRA HELP ✔️ expected
2	70	80	FALSE	FALSE	ON TRACK — intended: is exactly 70% "safe"?
3	80	50	FALSE	FALSE	ON TRACK — intended: is exactly

Step	attendanceRate	assignmentAverage	attendanceRate<70?	assignmentAverage<50?	flag result
					50 avg a pass?
4	80	49	FALSE	TRUE	NEEDS EXTRA HELP ✔️ expected

Diagnosis: the code isn't crashing, but rows 2 and 3 show the exact edge is currently treated as "safe." We confirmed with the team that 70% attendance and a 50 average should still count as the minimum PASS, so treating them as "on track" is correct and intentional — this trace just proves we checked the boundary instead of assuming it, and documents why no change was made.

Outcome: No code change needed — boundary confirmed correct and now has written evidence.

Summary of debugging evidence

#	Issue	Tool used	Status
1	Late submissions get no colour	Trace table	Fixed & retested ✔️
2	failedAttempts never resets	Trace table	Fixed & retested ✔️
3	Boundary values (70 / 50)	Trace table (boundary test)	Verified correct, documented ✔️